CRIPTA

Plan de trabajo y división del proyecto

Estructuras de Datos · II Ciclo 2026 · 2 de octubre de 2026

Este documento explica cómo nos vamos a repartir el proyecto. Ya existe un **esqueleto del código** con todas las clases y funciones definidas, pero sin implementar: cada una solo lanza `NotImplementedError`. Cada archivo dice en su primera línea a quién le toca: `[Persona A]`, `[Persona B]` o `[CONTRATO A↔B]` (compartido).

La idea es que cada uno pueda avanzar **sin esperar al otro**. Las dos mitades solo se tocan en unos pocos archivos de contrato que ya están definidos. Falta decidir quién toma cada mitad.

1. La división en una tabla

	Persona A — Motor del juego	Persona B — Datos, red y persistencia
Idea	Todo lo que pasa **dentro** de la partida	Todo lo que **entra y sale** de la partida
Capas MVC	Modelo + Vista	Adaptadores + Controlador
Enunciado	§2 completo, §4.1, §4.5, §4.6, §4.7, §4.8, §5.2, §10.2	§3, §4.2, §4.3, §4.4, §4.9, §4.10, §4.11, §5.1, §5.3, §6, §10.1
Rubros (§11)	Juego funcional (20 %), retroceso, rendimiento del ciclo de eventos	Cliente y presupuesto de red (25 %), caché, persistencia, mediciones
Documento de Decisiones	Secciones §4.1, §4.5, §4.6, §4.7, §4.8	Secciones §4.2, §4.3, §4.4, §4.9, §4.10, §4.11

2. Persona A — Motor del juego

Estructuras propias

- `estructuras/monticulo.py`: `MonticuloMin`, la base de la agenda de eventos.
- `estructuras/lista_doble.py`: `ListaDoble`, para el inventario. También la usa la caché LRU de B.
- `estructuras/buffer_circular.py`: `BufferCircular`, para los últimos 20 mensajes y los últimos 5 retrocesos.

Modelo

- `modelo/eventos.py`: `Agenda`, con cancelación y reprogramación de eventos.
- `modelo/actores.py`, `objetos.py`, `mapa.py`: jugador, enemigos, objetos, trampas, salas, puertas y rastro.
- `modelo/inventario.py`: cursor, soltar en O(1), mover al frente y vistas ordenadas.
- `modelo/efectos.py`: veneno, velocidad, antorcha, regeneración, trampas y cierre de puertas.
- `modelo/comportamiento.py`: guardián, errante y rastreador.
- `modelo/historial.py`: pergaminos de retroceso (bitácora de deshacer, sin copias del estado).
- `modelo/mensajes.py`: bitácora en pantalla.
- `modelo/juego.py`: la fachada del modelo y el bucle de eventos.
- `constantes.indice_direccion`.

Otros

- `vista/vista_consola.py`: la interfaz de consola.
- `bench/bench_modelo.py`: agenda, ciclo de §5.2 (meta: menos de 20 ms), inventario, historial, rastreadores y bitácora.

3. Persona B — Datos, red y persistencia

Estructuras propias

- `estructuras/tabla_hash.py`: `TablaHash` y `ConjuntoHash`, el reemplazo de `dict` en todo el proyecto.
- `estructuras/cola.py`: `ColaEnlazada`, para el BFS de precarga y las solicitudes pendientes en orden.
- `estructuras/ordenamiento.py`: inserción, mezcla, selector automático y búsqueda binaria.
- `estructuras/lru.py`: `CacheAcotada`, con desalojo y fijado de fichas en uso.

Contratos (la parte de B)

- `modelo/datos.py`: todos los `desde_json`.
- `modelo/acciones.py`: `a_linea` y `desde_linea` (formato del replay).

Adaptadores

- `adaptadores/fuente_datos.py`, `cliente_http.py`, `fuente_offline.py`: red con `X-Cripta-Client-Id`, manejo de 429 y presupuesto, y modo `--offline`.
- `adaptadores/almacen_disco.py`, `cache_catalogo.py`: caché persistente y resolución del catálogo por lotes.
- `adaptadores/guardado.py`: guardado binario propio, con JSON para comparar.
- `adaptadores/bitacora_replay.py`, `puntajes.py`.

Controlador

- `controlador/precarga.py`: precarga de salas por lotes, con orden determinista.
- `controlador/controlador.py`, `replay.py`, `aplicacion.py`: orquestación, `--replay` y banderas.

Otros

- `bench/runner.py`, `bench/sinteticos.py`, `bench/bench_datos.py`.
- `README.md`.

4. Tareas de los dos

- `PROMPTS_LLM.md`: cada quien agrega los prompts que use (§10.3).
- Revisar las secciones del Documento que escribió el otro. En la defensa **cualquiera** puede tener que explicar **cualquier** decisión (§9).
- Pruebas de sus propias estructuras. Se recomienda una carpeta `tests/` con `unittest`.

> Si una mitad avanza más rápido, la pieza más fácil de mover es la **vista de consola**: puede pasar de A a B sin tocar nada más.

5. Orden sugerido

Fase	Persona A	Persona B
0. Arranque (juntos)	Crear el repositorio en GitHub, subir el esqueleto y leer juntos los archivos de contrato.	
1. Estructuras base + pruebas	`MonticuloMin`, `ListaDoble`, `BufferCircular`, `indice_direccion`	`TablaHash`, `ConjuntoHash`, `ColaEnlazada`, los `desde_json` de `datos.py` y `FichasEnMemoria`
2. Núcleo por separado	`Agenda`, `Mapa`, actores, `Juego` (mover, atacar, esperar y el bucle), IA, rastro y una vista mínima. Se prueba con `datos_ejemplo/` + `FichasEnMemoria`, sin red.	`ClienteHTTP` (429 y presupuesto), `FuenteOffline`, almacenes en disco, `CacheAcotada`, `CacheCatalogo`, ordenamientos y `aplicacion.py`
3. Integración	Ajustar lo que pida B en `Mapa.vecinos`, `incorporar_contenido` y la API de `Juego`.	`Controlador`, `PlanificadorPrecarga` y `EscritorReplay`. Meta: jugar cripta-01 de principio a fin.
4. Completar	Inventario con vistas ordenadas, efectos, retroceso, `exportar_estado` y `desde_estado`.	Guardado binario + JSON, `ReproductorReplay`, puntajes, `runner` y `generar_cripta`.
5. Evidencia	`bench_modelo` + sus secciones del Documento.	`bench_datos` + sus secciones del Documento + README.

6. Qué necesita cada uno del otro

Conviene entregar estas piezas en la fase indicada para que el otro no se quede bloqueado.

Quién	Qué	Fase
A ← B	`TablaHash` (el modelo no puede usar `dict`)	1
A ← B	Los `desde_json` de `datos.py` y `FichasEnMemoria`, para probar con `datos_ejemplo/`	1
A ← B	`ordenamiento.ordenar` (vistas del inventario y orden de activación de enemigos)	2
B ← A	`ListaDoble` (la usa `CacheAcotada`)	1
B ← A	`Mapa.construir`, `Mapa.vecinos`, `Mapa.incorporar_contenido` y `Juego.ejecutar_accion`	3
B ← A	`Juego.exportar_estado` y `Juego.desde_estado`	4

7. Puntos de integración delicados

1. **Antes de mover.** El controlador pregunta `juego.sala_destino(dir)` y llama a `precarga.asegurar(sala)`, que trae el contenido y resuelve sus fichas. Recién entonces llama a `juego.ejecutar_accion`. Si el modelo responde `requiere_sala`, se asegura esa sala y se reintenta.
2. **Fijar y liberar fichas.** A decide *cuándo* (entrar o salir de una sala, recoger, soltar, morir, retroceder) y B decide *cómo* (`CacheCatalogo.fijar` / `liberar`). Si esto queda incoherente, la caché desaloja fichas en uso.
3. **Retroceso.** Toda mutación del estado del juego pasa por `Juego._asignar` (u otro helper que registre un `Cambio`). Los pergaminos y la caché quedan fuera del retroceso.
4. **Determinismo** (sin esto el replay no funciona): toda aleatoriedad usa `juego.azar`; las salidas se recorren siempre en orden N, S, E, O; nunca se itera una `TablaHash` para decidir algo del juego, porque `hash(str)` cambia

entre ejecuciones; las respuestas HTTP se incorporan en el orden en que se pidieron y no consumen azar ni números de secuencia; los ids de los objetos se generan de forma determinista.

5. **Contratos.** Los archivos `[CONTRATO A↔B]` (`datos.py`, `acciones.py`, `puertos.py`, `guardable.py`, `comandos.py` y `constantes.py`) se pueden cambiar, pero **avisando** y en un commit aparte.

8. Reglas para todo el código (§7, §9)

- Nada de `dict` como índice, caché o tabla de búsqueda: se usa `TablaHash` / `ConjuntoHash`. Leer el resultado de `respuesta.json()` sí está permitido.
- Nada de `sorted()` ni `list.sort()`: se usa `estructuras.ordenamiento.ordenar`.
- No se escriben en el código salas, ids, páginas, presupuestos ni estadísticas de una cripta.
- El modelo no importa `requests`, `json` ni `os`, no usa `open()` y no conoce la vista.
- Cada decisión de estructura se mide con `--bench` **mientras se implementa**, no al final. Es lo que pide el Documento de Decisiones (§8), incluida al menos una alternativa descartada medida.

9. Lo que dice el API real (revisado el 2 de octubre)

- Por ahora solo está publicada `cripta-01`: 6 salas, presupuesto de 9 solicitudes e inventario de 10. Sus respuestas están guardadas en `datos_ejemplo/cripta-01/`.
- Las salidas son **dirigidas**: `2 →E→ 5` está cerrada con llave, pero `5 →O→ 2` no declara cierre. Hay que decidir si la puerta se comparte entre los dos sentidos (ver `modelo/mapa.py`).
- El enemigo `e-401` viene sin `vida`, así que se usa `vida_max`.
- Los objetos del contenido son solo ids de tipo. El modelo tiene que generar los ids de instancia.
- `abre` de una llave (`"puerta_2_7"`) no coincide con las salas. La regla del juego es que la salida se abre con la llave cuyo id coincide con `salida.llave`.
- El esquema de fichas **no** tiene veneno ni regeneración, aunque §2.10 los menciona. Se leen como opcionales.
- Sin el encabezado `X-Cripta-Client-Id`, el servidor responde **400**.

10. Primeros pasos

1. Crear el repositorio en GitHub y subir el esqueleto.
2. Instalar Python 3.11 o superior y correr `pip install -r requirements.txt`.
3. Leer juntos los archivos de contrato y el `PLAN_DE_TRABAJO.md` del repositorio.
4. Decidir quién es Persona A y quién Persona B.
5. Arrancar la Fase 1: cada uno sus estructuras base, con pruebas, porque el otro las necesita pronto.